Untitled60

April 9, 2026

[1]:
```
%pip install -q numpy pandas matplotlib seaborn scikit-learn
```

```
Note: you may need to restart the kernel to use updated packages.
```

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold, cross_validate,␣
↪GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor,␣
↪HistGradientBoostingRegressor

from sklearn.metrics import (
    mean_absolute_error,
    root_mean_squared_error,
    r2_score
)
```

[7]:
```
# configurations
pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", lambda x: f"{x:.3f}")
sns.set_theme(style="darkgrid")

plt.rcParams.update({
    "axes.titlesize": 10,
    "axes.labelsize": 9,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8
})
```

```
RANDOM_STATE = 42
CSV_PATH = "housing.csv"          # update path for a different dataset
TARGET_COL = "median_house_value"
```

[8]:
```
df = pd.read_csv(CSV_PATH)
```

[9]:
```
print("DataFrame shape:", df.shape)
```

```
DataFrame shape: (20640, 10)
```

[10]:
```
df.head()
```

[10]:
```
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0   -122.230    37.880              41.000      880.000         129.000
1   -122.220    37.860              21.000     7099.000        1106.000
2   -122.240    37.850              52.000     1467.000         190.000
3   -122.250    37.850              52.000     1274.000         235.000
4   -122.250    37.850              52.000     1627.000         280.000

   population  households  median_income  median_house_value ocean_proximity
0     322.000     126.000          8.325          452600.000        NEAR BAY
1    2401.000    1138.000          8.301          358500.000        NEAR BAY
2     496.000     177.000          7.257          352100.000        NEAR BAY
3     558.000     219.000          5.643          341300.000        NEAR BAY
4     565.000     259.000          3.846          342200.000        NEAR BAY
```

[11]:
```
df.columns
```

[11]:
```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income',
       'median_house_value', 'ocean_proximity'],
      dtype='object')
```

[12]:
```
# basic dataset overview
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
```

```
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

[13]:
```python
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = df.select_dtypes(include=["object"]).columns.tolist()

print("Target column:", TARGET_COL)
print("Numerical columns:", num_cols)
print("Categorical columns:", cat_cols)
```

```
Target column: median_house_value
Numerical columns: ['longitude', 'latitude', 'housing_median_age',
'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income',
'median_house_value']
Categorical columns: ['ocean_proximity']
```

[14]:
```python
# missing values analysis
print("\nMissing values per column:")
print(df.isna().sum())
```

```
Missing values per column:
longitude               0
latitude                0
housing_median_age      0
total_rooms             0
total_bedrooms        207
population              0
households              0
median_income           0
median_house_value      0
ocean_proximity         0
dtype: int64
```

[15]:
```python
# check presence of encoded missing values
for col in df.columns:
    print(df[col].value_counts().head(20))
```

```
longitude
-118.310    162
-118.300    160
-118.290    148
-118.270    144
-118.320    142
-118.280    141
-118.350    140
-118.360    138
```

```
-118.190    135
-118.370    128
-118.250    128
-118.200    126
-118.140    125
-118.130    121
-118.260    121
-118.180    120
-118.340    119
-118.210    118
-118.150    116
-118.120    112
Name: count, dtype: int64
latitude
34.060    244
34.050    236
34.080    234
34.070    231
34.040    221
34.090    212
34.020    208
34.100    203
34.030    193
33.930    181
33.940    175
33.970    172
33.990    168
33.880    164
34.110    162
33.980    162
34.160    159
34.120    158
34.150    157
34.010    156
Name: count, dtype: int64
housing_median_age
52.000    1273
36.000     862
35.000     824
16.000     771
17.000     698
34.000     689
26.000     619
33.000     615
18.000     570
25.000     566
32.000     565
37.000     537
```

```
15.000     512
19.000     502
27.000     488
24.000     478
30.000     476
28.000     471
20.000     465
29.000     461
Name: count, dtype: int64
total_rooms
1527.000    18
1613.000    17
1582.000    17
2127.000    16
1717.000    15
2053.000    15
1607.000    15
1722.000    15
1471.000    15
1703.000    15
1724.000    14
1743.000    14
1745.000    14
1787.000    14
1562.000    14
1880.000    14
1731.000    14
1705.000    14
1650.000    14
2228.000    13
Name: count, dtype: int64
total_bedrooms
280.000    55
331.000    51
345.000    50
343.000    49
393.000    49
328.000    48
348.000    48
394.000    48
272.000    47
309.000    47
314.000    46
322.000    46
399.000    46
295.000    46
317.000    46
313.000    45
```

```
290.000    45
346.000    45
340.000    45
287.000    45
Name: count, dtype: int64
population
891.000     25
761.000     24
1227.000    24
1052.000    24
850.000     24
825.000     23
782.000     22
999.000     22
1005.000    22
753.000     21
781.000     21
1098.000    21
872.000     21
1155.000    20
1301.000    20
1056.000    20
1054.000    20
899.000     20
1011.000    20
735.000     20
Name: count, dtype: int64
households
306.000    57
386.000    56
335.000    56
282.000    55
429.000    54
375.000    53
284.000    51
297.000    51
278.000    50
340.000    50
362.000    50
380.000    50
316.000    49
329.000    49
319.000    49
330.000    49
309.000    48
426.000    48
377.000    48
341.000    48
```

```
Name: count, dtype: int64
median_income
3.125     49
15.000    49
2.875     46
2.625     44
4.125     44
3.875     41
3.375     38
3.000     38
4.000     37
3.625     37
4.375     35
2.125     33
2.375     32
4.625     31
3.500     30
3.750     29
3.250     29
4.875     29
2.250     29
1.625     29
Name: count, dtype: int64
median_house_value
500001.000    965
137500.000    122
162500.000    117
112500.000    103
187500.000     93
225000.000     92
350000.000     79
87500.000      78
275000.000     65
150000.000     64
175000.000     63
100000.000     62
125000.000     56
67500.000      55
250000.000     47
200000.000     46
118800.000     39
450000.000     37
156300.000     35
212500.000     33
Name: count, dtype: int64
ocean_proximity
<1H OCEAN     9136
INLAND        6551
```

```
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: count, dtype: int64
```

[16]:
```python
# duplicates
duplicate_mask = df.duplicated()
num_duplicates = duplicate_mask.sum()
print("Number of duplicate rows:", num_duplicates)

# (optional) drop duplicates if present
# df = df.drop_duplicates()
# print("Shape after dropping duplicates:", df.shape)
```

```
Number of duplicate rows: 0
```

[17]:
```python
# descriptive stat
df[num_cols].describe()
```

[17]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
count	20640.000	20640.000	20640.000	20640.000	20433.000	
mean	-119.570	35.632	28.639	2635.763	537.871	
std	2.004	2.136	12.586	2181.615	421.385	
min	-124.350	32.540	1.000	2.000	1.000	
25%	-121.800	33.930	18.000	1447.750	296.000	
50%	-118.490	34.260	29.000	2127.000	435.000	
75%	-118.010	37.710	37.000	3148.000	647.000	
max	-114.310	41.950	52.000	39320.000	6445.000	

	population	households	median_income	median_house_value
count	20640.000	20640.000	20640.000	20640.000
mean	1425.477	499.540	3.871	206855.817
std	1132.462	382.330	1.900	115395.616
min	3.000	1.000	0.500	14999.000
25%	787.000	280.000	2.563	119600.000
50%	1166.000	409.000	3.535	179700.000
75%	1725.000	605.000	4.743	264725.000
max	35682.000	6082.000	15.000	500001.000

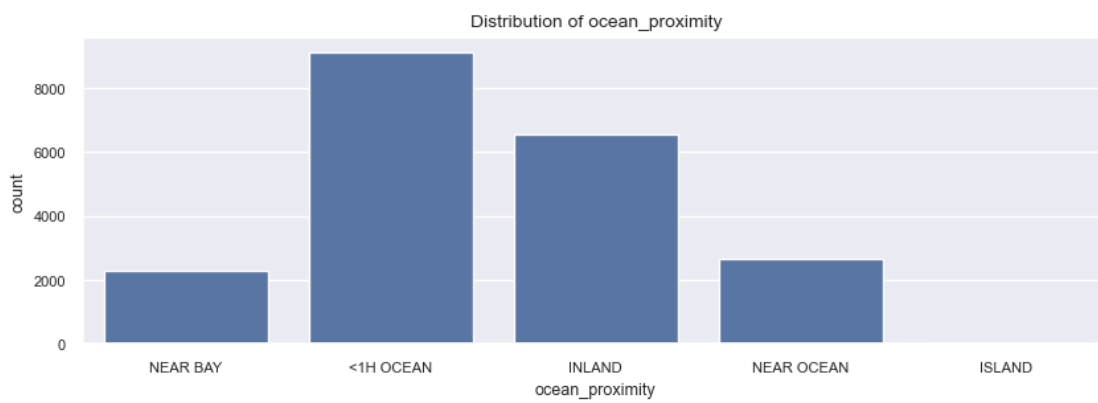
[18]:
```python
# descriptive stat
df[num_cols].describe().T
```

[18]:

	count	mean	std	min	25%	\
longitude	20640.000	-119.570	2.004	-124.350	-121.800	
latitude	20640.000	35.632	2.136	32.540	33.930	
housing_median_age	20640.000	28.639	12.586	1.000	18.000	
total_rooms	20640.000	2635.763	2181.615	2.000	1447.750	
total_bedrooms	20433.000	537.871	421.385	1.000	296.000	

```
population         20640.000   1425.477   1132.462     3.000    787.000
households         20640.000    499.540    382.330     1.000    280.000
median_income      20640.000      3.871      1.900     0.500      2.563
median_house_value 20640.000 206855.817 115395.616 14999.000 119600.000

                          50%        75%        max
longitude            -118.490   -118.010   -114.310
latitude               34.260     37.710     41.950
housing_median_age     29.000     37.000     52.000
total_rooms          2127.000   3148.000  39320.000
total_bedrooms        435.000    647.000   6445.000
population           1166.000   1725.000  35682.000
households            409.000    605.000   6082.000
median_income           3.535      4.743     15.000
median_house_value 179700.000 264725.000 500001.000
```

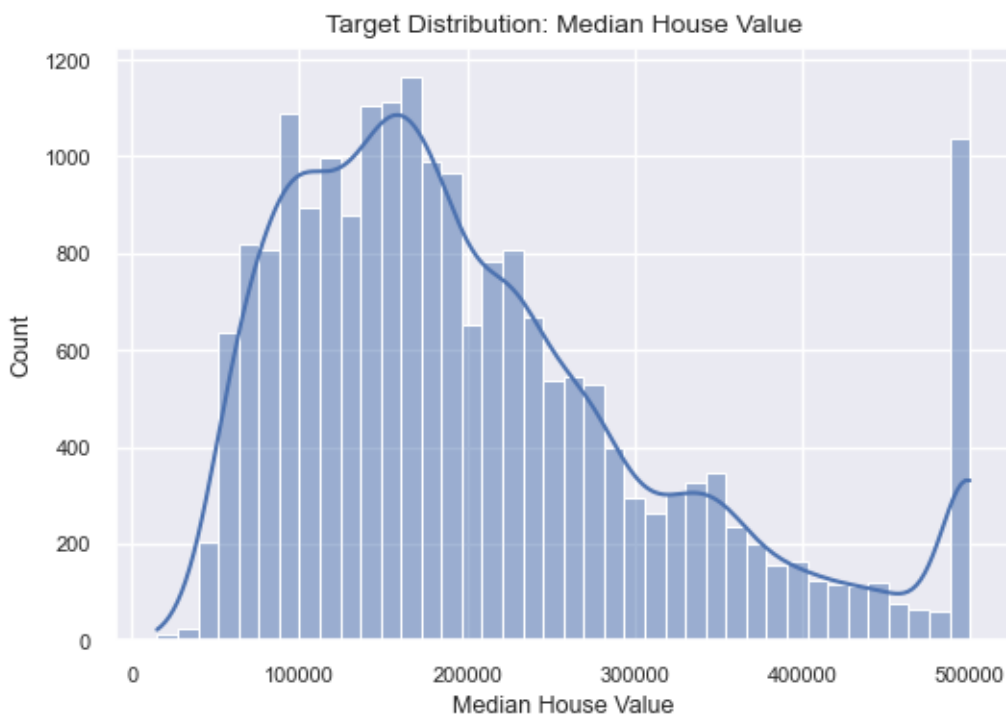
[19]:
```python
# countplot for categorical columns
for col in cat_cols:
    plt.figure(figsize=(10, 3))
    sns.countplot(x=col, data=df)
    plt.title(f"Distribution of {col}")
    plt.show()
```



[20]:
```python
for col in cat_cols:
    print(df[col].value_counts())
```

```
ocean_proximity
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: count, dtype: int64
```

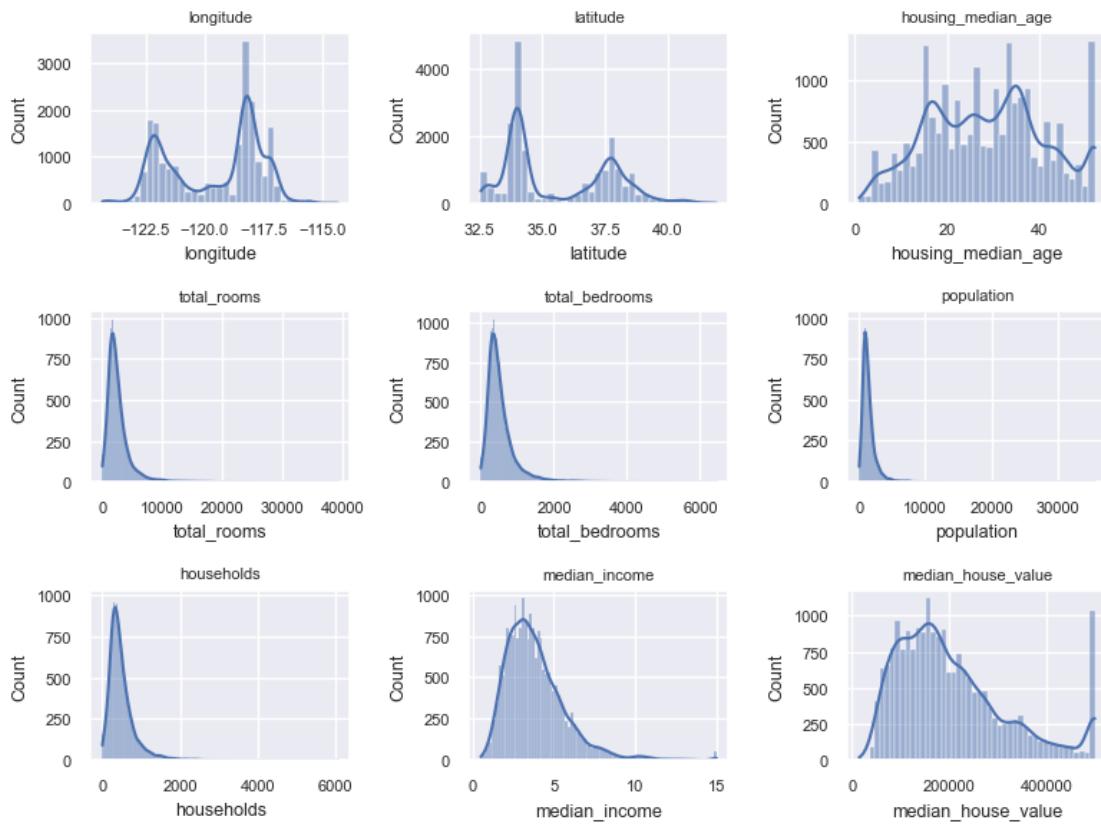
[21]:
```
# target column distribution
plt.figure(figsize=(6,4))
sns.histplot(df[TARGET_COL], bins=40, kde=True)
plt.title("Target Distribution: Median House Value")
plt.xlabel("Median House Value")
plt.show()
```



[22]:
```
df[TARGET_COL].value_counts()
# higher cap
```

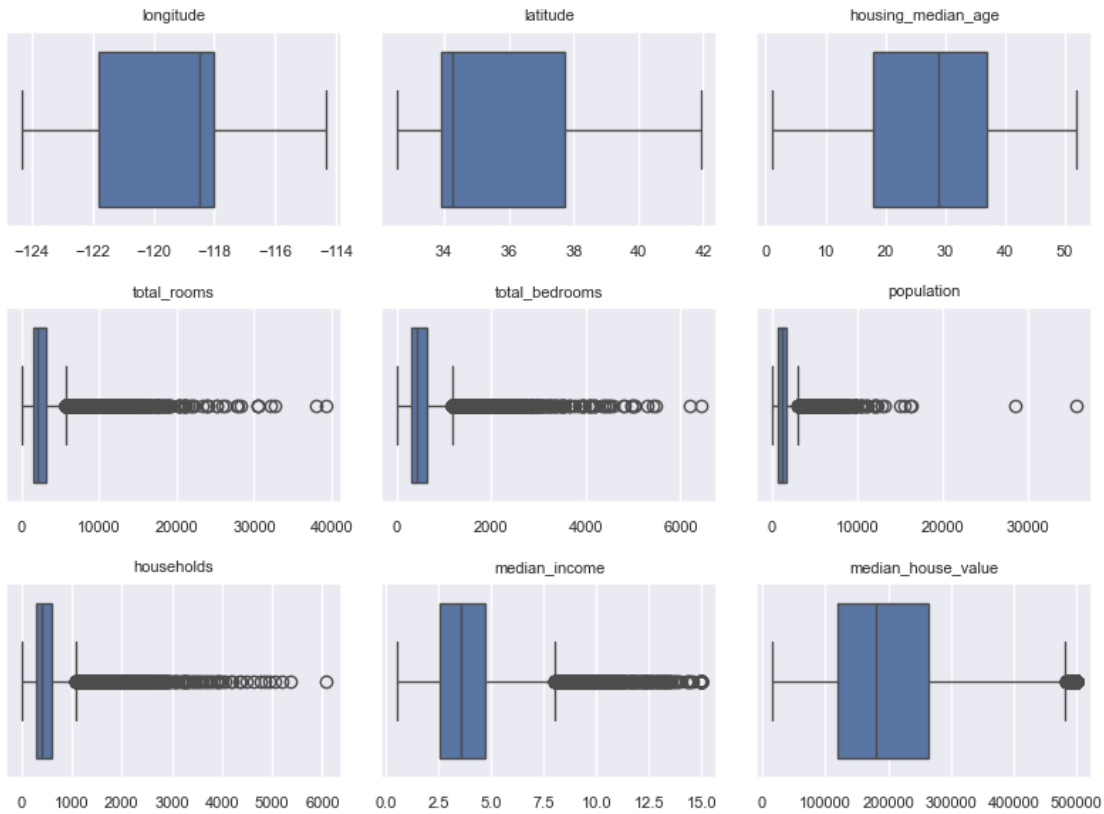
[22]:
```
median_house_value
500001.000    965
137500.000    122
162500.000    117
112500.000    103
187500.000     93
              …
359200.000      1
54900.000       1
377600.000      1
81200.000       1
47000.000       1
Name: count, Length: 3842, dtype: int64
```

[23]:
```
# histogram plot - distribution
fig, axes = plt.subplots(3, 3, figsize=(8, 6))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    sns.histplot(df[col], kde=True, ax=axes[i])
    axes[i].set_title(col, fontsize=8)

plt.tight_layout()
plt.show()
```

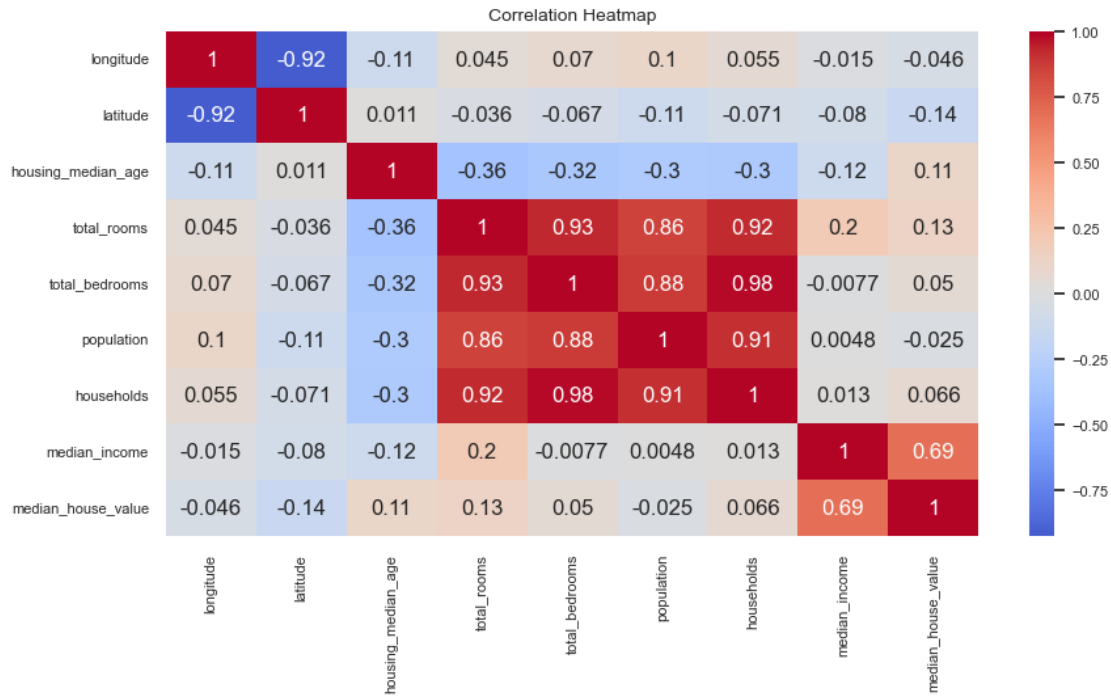


[24]:
```
# outliers analysis - boxplot
fig, axes = plt.subplots(3, 3, figsize=(8, 6))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    sns.boxplot(x=df[col], ax=axes[i])
    axes[i].set_title(col, fontsize=8)
    axes[i].set_xlabel("")

plt.tight_layout()
```

```
plt.show()
```



[25]:
```
# identify presence of highly correlated columns & feature relationships
plt.figure(figsize=(10, 5))
sns.heatmap(
    df[num_cols].corr(),
    annot=True,
    cmap="coolwarm",
    center=0
)
plt.title("Correlation Heatmap")
plt.show()
```



[26]:
```
# Correlation with target
corr_with_target = df[num_cols].corr()[TARGET_COL].sort_values(ascending=False)
print("\nCorrelation with target:")
print(corr_with_target)
```

```
Correlation with target:
median_house_value    1.000
median_income         0.688
total_rooms           0.134
housing_median_age    0.106
households            0.066
total_bedrooms        0.050
population           -0.025
longitude            -0.046
latitude             -0.144
Name: median_house_value, dtype: float64
```

[27]:
```
# Data Preprocessing
# separate features and target
X = df.drop(columns=[TARGET_COL])
y = df[TARGET_COL]
```

[28]:
```
X.head()
```

[28]:
```
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0   -122.230    37.880              41.000      880.000         129.000
1   -122.220    37.860              21.000     7099.000        1106.000
2   -122.240    37.850              52.000     1467.000         190.000
3   -122.250    37.850              52.000     1274.000         235.000
4   -122.250    37.850              52.000     1627.000         280.000

   population  households  median_income ocean_proximity
0     322.000     126.000          8.325        NEAR BAY
1    2401.000    1138.000          8.301        NEAR BAY
2     496.000     177.000          7.257        NEAR BAY
3     558.000     219.000          5.643        NEAR BAY
4     565.000     259.000          3.846        NEAR BAY
```

[29]:
```python
y.head()
```

[29]:
```
0   452600.000
1   358500.000
2   352100.000
3   341300.000
4   342200.000
Name: median_house_value, dtype: float64
```

[30]:
```python
# train test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE
)
```

[31]:
```python
print("Train shape:", X_train.shape)

print("Test shape:", X_test.shape)
```

```
Train shape: (16512, 9)
Test shape: (4128, 9)
```

[32]:
```python
numerical_features = X_train.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X_train.select_dtypes(exclude=[np.number]).columns.
 ↪tolist()

print("Numerical features:", numerical_features)
print("Categorical features:", categorical_features)

# numerical features - preprocessing steps
numerical_transformer = Pipeline(
    steps=[
```

```
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)

# categorical features - preprocessing steps
categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]
)

# preprocessing pipeline
preprocess = ColumnTransformer(
    transformers=[
        ("num", numerical_transformer, numerical_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

```
Numerical features: ['longitude', 'latitude', 'housing_median_age',
'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income']
Categorical features: ['ocean_proximity']
```

[33]:
```
# Baseline model (No CV, No Tuning)
baseline_pipe = Pipeline(
    steps=[
        ("preprocess", preprocess),
        ("model", LinearRegression())
    ]
)
```

[34]:
```
# preprocess the data and train the baseline model
baseline_pipe.fit(X_train, y_train)
```

[34]:
```
Pipeline(steps=[('preprocess',
                 ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
                                                                  ('scaler',
StandardScaler())]),
                                                  ['longitude', 'latitude',
                                                   'housing_median_age',
                                                   'total_rooms',
                                                   'total_bedrooms',
                                                   'population', 'households',
                                                   'median_income']),
```

```
                                                        ('cat',
                                                         Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
                                                                         ('onehot',
OneHotEncoder(handle_unknown='ignore'))]),
                                                         ['ocean_proximity'])])),
                 ('model', LinearRegression())])
```

[35]:
```
train_baseline_pred = baseline_pipe.predict(X_train)
test_baseline_pred = baseline_pipe.predict(X_test)
```

[36]:
```
train_baseline_pred[:5]
```

[36]:
```
array([188628.0772436 , 290379.8948687 , 250985.48476349, 146878.07878194,
       165789.41368924])
```

[37]:
```
y_train[:5]
```

[37]:
```
14196   103000.000
8267    382100.000
17445   172600.000
14265    93400.000
2271     96500.000
Name: median_house_value, dtype: float64
```

[38]:
```
train_baseline_rmse = root_mean_squared_error(y_train, train_baseline_pred)
train_baseline_mae = mean_absolute_error(y_train, train_baseline_pred)
train_baseline_r2 = r2_score(y_train, train_baseline_pred)

print("\n=== TRAIN BASELINE METRICS (LinearRegression) ===")
print(f"RMSE: {train_baseline_rmse:.3f}")
print(f"MAE : {train_baseline_mae:.3f}")
print(f"R2  : {train_baseline_r2:.3f}")
```

```
=== TRAIN BASELINE METRICS (LinearRegression) ===
RMSE: 68433.937
MAE : 49594.842
R2  : 0.650
```

[39]:
```
#  Model Selection & Optimization
# models to try
models = {
    "LinearRegression": LinearRegression(),
    "Ridge": Ridge(random_state=RANDOM_STATE),
    "Lasso": Lasso(random_state=RANDOM_STATE, max_iter=10000),
    "RandomForest": RandomForestRegressor(),
    "HistGB": HistGradientBoostingRegressor()
```

```
}
```

[40]:
```
k = 5
cv = KFold(n_splits=k, shuffle=True, random_state=RANDOM_STATE)
```

[41]:
```
scoring = {
    "rmse": "neg_root_mean_squared_error",
    "mae": "neg_mean_absolute_error",
    "r2": "r2"
}
```

[62]:
```
rows = []

for name, model in models.items():
    pipe = Pipeline(
        steps=[
            ("preprocess", preprocess),
            ("model", model)
        ]
    )
    scores = cross_validate(pipe, X_train, y_train, cv=cv, scoring=scoring,␣
 ↪n_jobs=1)
    rows.append({
        "model": name,
        "cv_rmse": -scores["test_rmse"].mean(),
        "cv_mae": -scores["test_mae"].mean(),
        "cv_r2": scores["test_r2"].mean()
    })


# sort based on the lowest RMSE value
cv_results = pd.DataFrame(rows).sort_values("cv_rmse")
print("=== CV Model Comparison ===")
print(cv_results)
```

```
=== CV Model Comparison ===
              model   cv_rmse    cv_mae  cv_r2
4            HistGB 48267.359 32367.685  0.826
3      RandomForest 49362.771 32220.373  0.818
1             Ridge 68595.617 49664.331  0.648
2             Lasso 68603.233 49667.263  0.648
0  LinearRegression 68604.163 49667.159  0.648
```

[63]:
```
cv_results
```

[63]:
```
              model   cv_rmse    cv_mae  cv_r2
4            HistGB 48267.359 32367.685  0.826
3      RandomForest 49362.771 32220.373  0.818
```

```
1             Ridge 68595.617 49664.331  0.648
2             Lasso 68603.233 49667.263  0.648
0  LinearRegression 68604.163 49667.159  0.648
```

[65]:
```python
best_row = cv_results.sort_values("cv_rmse").iloc[0]

best_model_name = best_row["model"]
best_rmse = best_row["cv_rmse"]

print("Best model based on CV RMSE:")
print("Model :", best_model_name)
print("CV RMSE:", best_rmse)
```

```
Best model based on CV RMSE:
Model : HistGB
CV RMSE: 48267.35933399352
```

[66]:
```python
#BEST MODEL: HistGradientBoostingRegressor

# 7. Hyperparameter Tuning
```

[67]:
```python
hgb_pipe = Pipeline(
    steps=[
        ("preprocess", preprocess),
        ("model", HistGradientBoostingRegressor(random_state=RANDOM_STATE))
    ]
)
```

[68]:
```python
# hyperparameters combination
param_grid = {
    "model__learning_rate": [0.03, 0.05, 0.1],
    "model__max_depth": [None, 3, 6],
    "model__max_leaf_nodes": [15, 31, 63],
    "model__min_samples_leaf": [20, 50, 100],
    "model__l2_regularization": [0.0, 0.1, 1.0]
}
```

[69]:
```python
grid = GridSearchCV(
    estimator=hgb_pipe,
    param_grid=param_grid,
    cv=cv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    verbose=1,
)
```

[70]:
```python
# perform grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 243 candidates, totalling 1215 fits
```

[70]:
```
GridSearchCV(cv=KFold(n_splits=5, random_state=42, shuffle=True),
             estimator=Pipeline(steps=[('preprocess',
                                        ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
          SimpleImputer(strategy='median')),
         ('scaler',
          StandardScaler())]),
['longitude',
'latitude',
'housing_median_age',
'total_rooms',
'total_bedrooms',
'population',
'households',
'median_income'…
          OneHotEncoder(handle_unknown='ignore'))]),
['ocean_proximity'])])),
                                       ('model',
HistGradientBoostingRegressor(random_state=42))]),
             n_jobs=-1,
             param_grid={'model__l2_regularization': [0.0, 0.1, 1.0],
                         'model__learning_rate': [0.03, 0.05, 0.1],
                         'model__max_depth': [None, 3, 6],
                         'model__max_leaf_nodes': [15, 31, 63],
                         'model__min_samples_leaf': [20, 50, 100]},
             scoring='neg_root_mean_squared_error', verbose=1)
```

[97]:
```python
# Step 1: Import necessary libraries if not already imported
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Step 2: Identify categorical columns in your dataset
# Assuming 'NEAR OCEAN' is in a column like 'ocean_proximity'
# Replace this with your actual categorical column names
categorical_cols = ['ocean_proximity']  # Add all categorical columns here
numerical_cols = [col for col in X_train.columns if col not in categorical_cols]

# Step 3: Create a preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ])
```

```python
# Step 4: Create a pipeline with preprocessing and the model
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', hgb_best)  # Your existing model
])

# Step 5: Fit the pipeline and make predictions
pipeline.fit(X_train, y_train)
train_final_pred = pipeline.predict(X_train)

# The rest of your code remains the same
train_final_rmse = root_mean_squared_error(y_train, train_final_pred)
train_final_mae = mean_absolute_error(y_train, train_final_pred)
train_final_r2 = r2_score(y_train, train_final_pred)

print("\n=== FINAL MODEL (Tuned HGB) Train Performance ===")
print(f"RMSE: {train_final_rmse:.3f}")
print(f"MAE : {train_final_mae:.3f}")
print(f"R2  : {train_final_r2:.3f}")
```

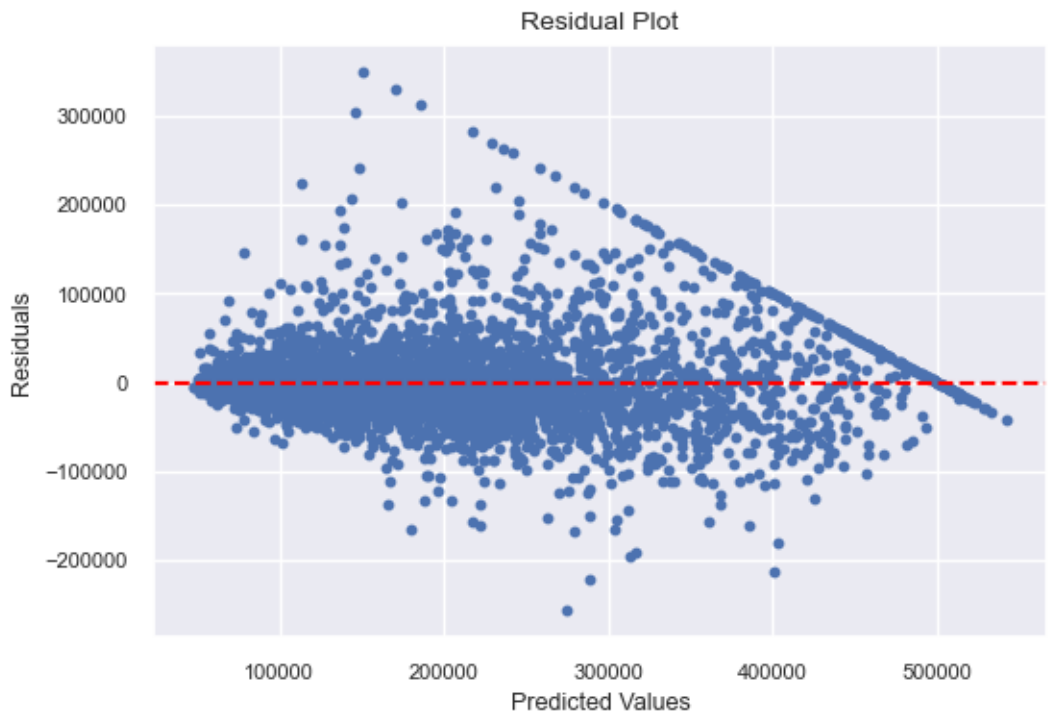
```
=== FINAL MODEL (Tuned HGB) Train Performance ===
RMSE: 41284.028
MAE : 28198.374
R2  : 0.873
```

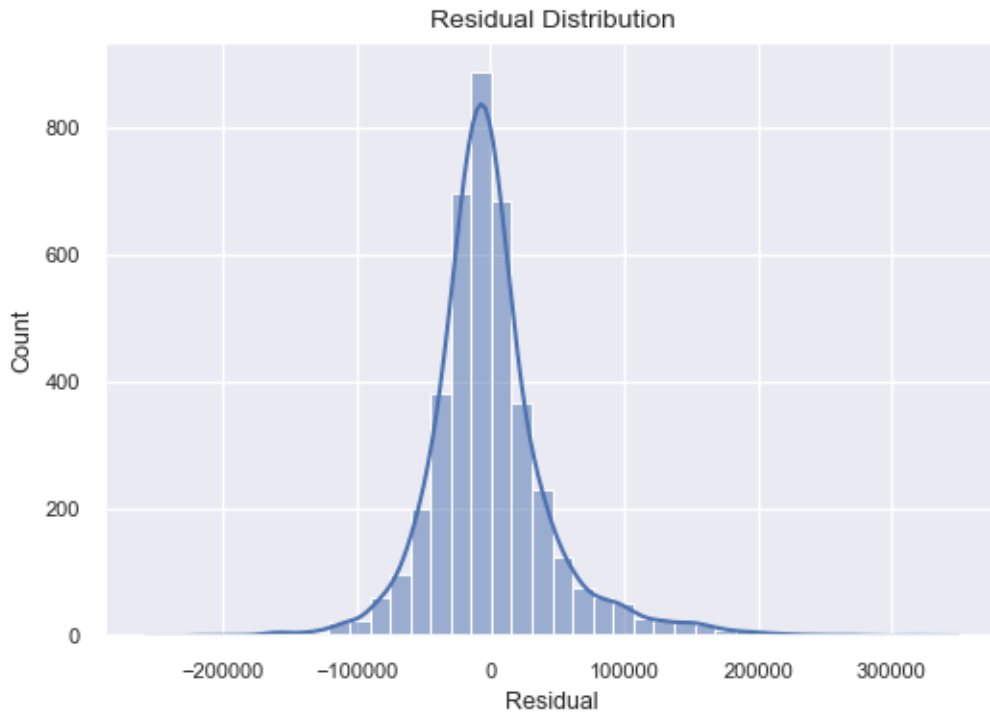
[104]:
```python
test_final_pred = pipeline.predict(X_test)

test_final_rmse = root_mean_squared_error(y_test, test_final_pred)
test_final_mae = mean_absolute_error(y_test, test_final_pred)
test_final_r2 = r2_score(y_test, test_final_pred)

print("\n=== FINAL MODEL (Tuned HGB) Test Performance ===")
print(f"RMSE: {test_final_rmse:.3f}")
print(f"MAE : {test_final_mae:.3f}")
print(f"R2  : {test_final_r2:.3f}")
```

```
=== FINAL MODEL (Tuned HGB) Test Performance ===
RMSE: 47372.608
MAE : 31613.791
R2  : 0.829
```

[106]:
```python
test_final_pred = pipeline.predict(X_test)
residuals = y_test - test_final_pred
plt.figure(figsize=(6, 4))
plt.scatter(test_final_pred, residuals, s=10)
plt.axhline(y=0, color='red', linestyle='--')
```

```
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()
plt.figure(figsize=(6, 4))
sns.histplot(residuals, bins=40, kde=True)
plt.title("Residual Distribution")
plt.xlabel("Residual")
plt.ylabel("Count")
plt.show()
```





[84]:
```
# Building a predictive system
```

[107]:
```
def predict_house_price(
    model,
    longitude: float,
    latitude: float,
    housing_median_age: float,
    total_rooms: float,
    total_bedrooms: float,
    population: float,
    households: float,
    median_income: float,
    ocean_proximity: str
) -> float:
    """
    Predict median_house_value for one new house.
    total_bedrooms can be np.nan (pipeline will impute).
    """
    new_row = pd.DataFrame([{
        "longitude": longitude,
        "latitude": latitude,
        "housing_median_age": housing_median_age,
        "total_rooms": total_rooms,
```

```
        "total_bedrooms": total_bedrooms,
        "population": population,
        "households": households,
        "median_income": median_income,
        "ocean_proximity": ocean_proximity
    }])

    return float(model.predict(new_row)[0])
```

[112]:
```
example_pred = predict_house_price(
    model=pipeline,   #  FIXED
    longitude=-122.230,
    latitude=37.880,
    housing_median_age=41,
    total_rooms=880,
    total_bedrooms=129,
    population=322,
    households=126,
    median_income=8.3252,
    ocean_proximity="NEAR BAY"
)

print("\nExample prediction:", round(example_pred, 2))
```

```
Example prediction: 448545.09
```

[ ]:
```

```